

Lecture 05 - Clipping

This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Source chunk	course_text_parts/03_lectures/05_clipping.txt
Extracted slide pages	56
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Untitled slide
Source cue	The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Professor-style explanation	For 'Untitled slide', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?
Page 2	Clipping Projected Geometry 1/2
Source cue	- Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized

Professor-style explanation	For 'Clipping Projected Geometry 1/2', put the formula aside for a moment and name the coordinate spaces. A transformation only makes sense when we know where the point, vector, or normal starts and where it should end. The slide gives us this anchor: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized. The professor-level way to read this slide is to narrate the movement: model space to world space, world to view, view to clip, or whichever step the slide is showing.
Technical commentary	This slide is about Clipping Projected Geometry 1/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Clipping Projected Geometry 1/2'?

Page 3	Clipping Projected Geometry 2/2
Source cue	- Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Clipping Projected Geometry 2/2' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Clipping Projected Geometry 2/2. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.

Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Clipping Projected Geometry 2/2'?
Page 4	5.1 Cohen-Sutherland Line Clipping
Source cue	5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping
Professor-style explanation	For '5.1 Cohen-Sutherland Line Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?
Page 5	5.1 Cohen-Sutherland Line Clipping
Source cue	Clip lines against rectangle
Professor-style explanation	For '5.1 Cohen-Sutherland Line Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: Clip lines against rectangle. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: Clip lines against rectangle
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?
Page 6	Brute-Force Line Clipping 1/2
Source cue	- Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary

Professor-style explanation	For 'Endpoint Analysis 1/2', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y)$, $\mathbf{q} = (x, y)$ and $\mathbf{r} = (x, y)$. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Endpoint Analysis 1/2. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y)$, $\mathbf{q} = (x, y)$ and $\mathbf{r} = (x, y)$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 1/2'?
Page 9	Endpoint Analysis 2/2
Source cue	- Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / $\mathbf{p}_x \geq x$ and $\mathbf{q}_x \leq x$ / and above y and below y , they are inside / $\mathbf{p}_y \geq y$ and $\mathbf{q}_y \leq y$
Professor-style explanation	For 'Endpoint Analysis 2/2', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / $\mathbf{p}_x \geq x$ and $\mathbf{q}_x \leq x$ / and above y and below y , they are inside / $\mathbf{p}_y \geq y$ and $\mathbf{q}_y \leq y$. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Endpoint Analysis 2/2. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / $\mathbf{p}_x \geq x$ and $\mathbf{q}_x \leq x$ / and above y and below y , they are inside / $\mathbf{p}_y \geq y$ and $\mathbf{q}_y \leq y$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 2/2'?
Page 10	Binary Coding of Half-Planes 1/3
Source cue	- Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode

Professor-style explanation	For 'Binary Coding of Half-Planes 1/3', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Binary Coding of Half-Planes 1/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Binary Coding of Half-Planes 1/3' into a causal sentence instead of repeating the slide title?
Page 11	Binary Coding of Half-Planes 2/3
Source cue	- Endpoint classification / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y) / 01 /$ - Outcodes: / 1001 1000 1010
Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Binary Coding of Half-Planes 2/3' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - Endpoint classification / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y) / 01 /$ - Outcodes: / 1001 1000 1010. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.
Technical commentary	This slide is about Binary Coding of Half-Planes 2/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Endpoint classification / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y) / 01 /$ - Outcodes: / 1001 1000 1010
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Binary Coding of Half-Planes 2/3'?
Page 12	Binary Coding of Half-Planes 3/3
Source cue	- Clipping, if $(oc0 \& oc1) = 0:$ / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip $\mathbf{p}$ at this clip edge

Professor-style explanation	For 'Binary Coding of Half-Planes 3/3', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Clipping, if $(oc0 \& oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip ■ at this clip edge. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Binary Coding of Half-Planes 3/3. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Clipping, if $(oc0 \& oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip ■ at this clip edge
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Binary Coding of Half-Planes 3/3'?

Page 13	Pseudo Code
Source cue	<pre>void cohensutherlandLineClipper(Line s, Rectangle r) { / int oc0 = outcode(s.p0, r); / int oc1 = outcode(s.p1, r); / bool done = false; / while (!done) { int outcode(Point p, Rectangle r) {</pre>
Professor-style explanation	For 'Pseudo Code', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: <pre>void cohensutherlandLineClipper(Line s, Rectangle r) { / int oc0 = outcode(s.p0, r); / int oc1 = outcode(s.p1, r); / bool done = false; / while (!done) { int outcode(Point p, Rectangle r) {</pre> . A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Pseudo Code. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: <pre>void cohensutherlandLineClipper(Line s, Rectangle r) { / int oc0 = outcode(s.p0, r); / int oc1 = outcode(s.p1, r); / bool done = false; / while (!done) { int outcode(Point p, Rectangle r) {</pre>
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?

Page 14	Example
Source cue	1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110

Professor-style explanation	For 'Example', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Example. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example' into a causal sentence instead of repeating the slide title?

Page 15	Evaluation
Source cue	- Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Professor-style explanation	For 'Evaluation', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Evaluation. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?

Page 16	5.2 Cyrus-Beck Line Clipping
Source cue	Clip lines against convex polygon
Professor-style explanation	For '5.2 Cyrus-Beck Line Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: Clip lines against convex polygon. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.

Technical commentary	This slide is about 5.2 Cyrus-Beck Line Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: Clip lines against convex polygon
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.2 Cyrus-Beck Line Clipping'?

Page 17	Cyrus-Beck Line Clipping
Source cue	- Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky
Professor-style explanation	For 'Cyrus-Beck Line Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Cyrus-Beck Line Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Cyrus-Beck Line Clipping'?

Page 18	Parametric Line Representation
Source cue	- Parametric line representation written as follows / $\mathbf{p} = \mathbf{p}_0 + \mathbf{d} \cdot t$, $0 \leq t \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 t 's for the 4 clip edges
Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Parametric Line Representation' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - Parametric line representation written as follows / $\mathbf{p} = \mathbf{p}_0 + \mathbf{d} \cdot t$, $0 \leq t \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 t 's for the 4 clip edges. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.

Technical commentary	This slide is about Parametric Line Representation. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Parametric line representation written as follows $\frac{y - y_1}{y_2 - y_1} = \frac{x - x_1}{x_2 - x_1}$, $0 \leq t \leq 1$ / - Proceeding / - Find the 4 t 's for the 4 clip edges
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Parametric Line Representation'?

Page 19	Intersection Point Calculation 1/2
Source cue	- Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (P - P_1)$ it can be decided / in which half-space a line point P lies / $N \cdot (P - P_1) < 0$ / $N \cdot (P - P_1) \geq 0$
Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Intersection Point Calculation 1/2' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (P - P_1)$ it can be decided / in which half-space a line point P lies / $N \cdot (P - P_1) < 0$ / $N \cdot (P - P_1) \geq 0$. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.
Technical commentary	This slide is about Intersection Point Calculation 1/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (P - P_1)$ it can be decided / in which half-space a line point P lies / $N \cdot (P - P_1) < 0$ / $N \cdot (P - P_1) \geq 0$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Intersection Point Calculation 1/2'?

Page 20	Intersection Point Calculation 2/2
Source cue	- Determine intersection by calculating t for $P_1 + t(P_2 - P_1) = P$ / - Substitution of t and regrouping gives $t = \frac{N \cdot (P - P_1)}{N \cdot (P_2 - P_1)}$
Professor-style explanation	For 'Intersection Point Calculation 2/2', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Determine intersection by calculating t for $P_1 + t(P_2 - P_1) = P$ / - Substitution of t and regrouping gives $t = \frac{N \cdot (P - P_1)}{N \cdot (P_2 - P_1)}$. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Intersection Point Calculation 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Determine intersection by calculating $\frac{y_1 - y_2}{x_2 - x_1}$ for $\frac{y_1 - y_2}{x_2 - x_1} = 0$ / - Substitution of x_1, y_1 and regrouping gives $\frac{y_1 - y_2}{x_2 - x_1} = 0$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Point Calculation 2/2' into a causal sentence instead of repeating the slide title?

Page 21	Determine Relevant t-Values
Source cue	- Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / $\frac{y_1 - y_2}{x_2 - x_1}$ / $\frac{y_1 - y_2}{x_2 - x_1}$ / $\frac{y_1 - y_2}{x_2 - x_1}$
Professor-style explanation	For 'Determine Relevant t-Values', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / $\frac{y_1 - y_2}{x_2 - x_1}$ / $\frac{y_1 - y_2}{x_2 - x_1}$ / $\frac{y_1 - y_2}{x_2 - x_1}$. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Determine Relevant t-Values. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / $\frac{y_1 - y_2}{x_2 - x_1}$ / $\frac{y_1 - y_2}{x_2 - x_1}$ / $\frac{y_1 - y_2}{x_2 - x_1}$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Determine Relevant t-Values' into a causal sentence instead of repeating the slide title?

Page 22	Pseudo Code
Source cue	<pre>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</pre>
Professor-style explanation	For 'Pseudo Code', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: <pre>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</pre> . A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Pseudo Code. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: <pre>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</pre>

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?
Page 23	Axis-Parallel Clipping Region
Source cue	- When clipping region is axis parallel and given by $[x_1, x_2] \times [y_1, y_2]$, / $\text{min}(x_1, x_2)$ / following values occur / $\text{min}(x_1, x_2) - \text{min}(x_1, x_2) / 0$
Professor-style explanation	For 'Axis-Parallel Clipping Region', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - When clipping region is axis parallel and given by $[x_1, x_2] \times [y_1, y_2]$, / $\text{min}(x_1, x_2)$ / following values occur / $\text{min}(x_1, x_2) - \text{min}(x_1, x_2) / 0$. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Axis-Parallel Clipping Region. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - When clipping region is axis parallel and given by $[x_1, x_2] \times [y_1, y_2]$, / $\text{min}(x_1, x_2)$ / following values occur / $\text{min}(x_1, x_2) - \text{min}(x_1, x_2) / 0$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Axis-Parallel Clipping Region'?
Page 24	Comparison with Cohen-Sutherland
Source cue	- Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected
Professor-style explanation	For 'Comparison with Cohen-Sutherland', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.

Technical commentary	This slide is about Comparison with Cohen-Sutherland. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Comparison with Cohen-Sutherland'?

Page 25	5.3 Sutherland-Hodgman Polygon Clipping
Source cue	A pipelined version of the scalar product algorithm extended to clip polygons
Professor-style explanation	For '5.3 Sutherland-Hodgman Polygon Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: A pipelined version of the scalar product algorithm extended to clip polygons. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about 5.3 Sutherland-Hodgman Polygon Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: A pipelined version of the scalar product algorithm extended to clip polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.3 Sutherland-Hodgman Polygon Clipping'?

Page 26	Sutherland-Hodgman Polygon Clipping 1/2
Source cue	- Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping
Professor-style explanation	For 'Sutherland-Hodgman Polygon Clipping 1/2', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.

Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 1/2. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 1/2'?

Page 27	Sutherland-Hodgman Polygon Clipping 2/2
Source cue	- Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region
Professor-style explanation	For 'Sutherland-Hodgman Polygon Clipping 2/2', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 2/2. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 2/2'?

Page 28	Four Cases
Source cue	- Depending on which case occurs, following points added to output list / - Case 1: ■ / - Case 2: Intersection point ■ / - Case 3: -- / - Case 4: Intersection point ■ and ■
Professor-style explanation	For 'Four Cases', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Depending on which case occurs, following points added to output list / - Case 1: ■ / - Case 2: Intersection point ■ / - Case 3: -- / - Case 4: Intersection point ■ and ■. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Four Cases. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Depending on which case occurs, following points added to output list / - Case 1: ■ / - Case 2: Intersection point ■ / - Case 3: -- / - Case 4: Intersection point ■ and ■
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Four Cases' into a causal sentence instead of repeating the slide title?

Page 29	Proceeding
Source cue	- Polygon to be clipped is defined by / - Vertices: [■ , ■ , ... , ■] / 1 2 ■ / - Contour: [■ , ■ , ... , ■ , ■] / 1 2 ■ 1
Professor-style explanation	For 'Proceeding', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Polygon to be clipped is defined by / - Vertices: [■ , ■ , ... , ■] / 1 2 ■ / - Contour: [■ , ■ , ... , ■ , ■] / 1 2 ■ 1. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Proceeding. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Polygon to be clipped is defined by / - Vertices: [■ , ■ , ... , ■] / 1 2 ■ / - Contour: [■ , ■ , ... , ■ , ■] / 1 2 ■ 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?

Page 30	Pipeline Version
Source cue	- Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware
Professor-style explanation	For 'Pipeline Version', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.

Technical commentary	This slide is about Pipeline Version. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pipeline Version'?

Page 31	Pseudo Code
Source cue	<pre>public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {</pre>
Professor-style explanation	For 'Pseudo Code', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: <pre>public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {</pre> . Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Pseudo Code. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: <pre>public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {</pre>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?

Page 32	Inside Test
Source cue	<pre>- Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top</pre>

Professor-style explanation	For 'Inside Test', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Inside Test. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Inside Test' into a causal sentence instead of repeating the slide title?

Page 33	Intersection Computation
Source cue	- For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;
Professor-style explanation	For 'Intersection Computation', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Intersection Computation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Computation' into a causal sentence instead of repeating the slide title?

Page 34	5.4 Weiler-Atherton Polygon Clipping
Source cue	Clip arbitrary polygons against arbitrary clip polygons
Professor-style explanation	For '5.4 Weiler-Atherton Polygon Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: Clip arbitrary polygons against arbitrary clip polygons. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.

Technical commentary	This slide is about 5.4 Weiler-Atherton Polygon Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: Clip arbitrary polygons against arbitrary clip polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.4 Weiler-Atherton Polygon Clipping'?

Page 35	Weiler-Atherton Polygon Clipping
Source cue	- Basic idea: edges of the clip polygon ■ and the polygon ■ to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Professor-style explanation	For 'Weiler-Atherton Polygon Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Basic idea: edges of the clip polygon ■ and the polygon ■ to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray). A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Weiler-Atherton Polygon Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Basic idea: edges of the clip polygon ■ and the polygon ■ to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Polygon Clipping'?

Page 36	Example Run
Source cue	- Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of ■ and ■ / - Double all edge segments, and label each segment / according to the adjacent region
Professor-style explanation	For 'Example Run', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of ■ and ■ / - Double all edge segments, and label each segment / according to the adjacent region. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Example Run. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of ■ and ■ / - Double all edge segments, and label each segment / according to the adjacent region
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example Run' into a causal sentence instead of repeating the slide title?

Page 37	Intersection Types
Source cue	- There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)
Professor-style explanation	For 'Intersection Types', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal). Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Intersection Types. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Types' into a causal sentence instead of repeating the slide title?

Page 38	Contour Processing
Source cue	- In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of ■ / and insert the contour edges of ■ one after the other
Professor-style explanation	For 'Contour Processing', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of ■ / and insert the contour edges of ■ one after the other. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Contour Processing. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of ■ / and insert the contour edges of ■ one after the other
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Contour Processing' into a causal sentence instead of repeating the slide title?

Page 39	Evaluation
Source cue	- Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion
Professor-style explanation	For 'Evaluation', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Evaluation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?

Page 40	5.5 Greiner-Hormann Polygon Clipping
Source cue	Exploitation of the winding number to clip arbitrary polygons
Professor-style explanation	For '5.5 Greiner-Hormann Polygon Clipping', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: Exploitation of the winding number to clip arbitrary polygons. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about 5.5 Greiner-Hormann Polygon Clipping. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: Exploitation of the winding number to clip arbitrary polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.5 Greiner-Hormann Polygon Clipping'?
Page 41	Greiner-Hormann
Source cue	- Basic Idea: clipped region is made up of all edges of ■ that lie in ■, / combined with all edges of ■ that lie in ■ / [Greiner & Hormann, TOG 1998]
Professor-style explanation	For 'Greiner-Hormann', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Basic Idea: clipped region is made up of all edges of ■ that lie in ■, / combined with all edges of ■ that lie in ■ / [Greiner & Hormann, TOG 1998]. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Greiner-Hormann. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Basic Idea: clipped region is made up of all edges of ■ that lie in ■, / combined with all edges of ■ that lie in ■ / [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Greiner-Hormann'?
Page 42	Chalk Wagon Metaphor
Source cue	- Using a chalk wagon metaphor / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■ / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■
Professor-style explanation	For 'Chalk Wagon Metaphor', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Using a chalk wagon metaphor / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■ / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Chalk Wagon Metaphor. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Using a chalk wagon metaphor / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■ / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Chalk Wagon Metaphor' into a causal sentence instead of repeating the slide title?

Page 43	Contour Detection
Source cue	- The edges of polygon P_1 within polygon P_2 can thus be found as follows / - Insert in P_2 all intersection points with P_1 / - Start at any vertex v of P_2 / - If v is within P_1 , the edge flag is turned on, otherwise not / - Traverse edges of P_2 and invert edge marker flag when intersecting with P_1
Professor-style explanation	For 'Contour Detection', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - The edges of polygon P_1 within polygon P_2 can thus be found as follows / - Insert in P_2 all intersection points with P_1 / - Start at any vertex v of P_2 / - If v is within P_1 , the edge flag is turned on, otherwise not / - Traverse edges of P_2 and invert edge marker flag when intersecting with P_1 . Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Contour Detection. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - The edges of polygon P_1 within polygon P_2 can thus be found as follows / - Insert in P_2 all intersection points with P_1 / - Start at any vertex v of P_2 / - If v is within P_1 , the edge flag is turned on, otherwise not / - Traverse edges of P_2 and invert edge marker flag when intersecting with P_1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Contour Detection' into a causal sentence instead of repeating the slide title?
Page 44	Set Operations
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	For 'Set Operations', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: [Greiner & Hormann, TOG 1998]. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Set Operations. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Set Operations'?
Page 45	Data Structure 1/2
Source cue	- Polygons exist as a closed, double-linked list / $P_1, P_2, P_3, \dots, P_n = P_1$ / 0 1 2 P_1 0 / - The following segments are included / $P_1 P_2, P_2 P_3, \dots, P_n P_1 = P_1 P_2$

Professor-style explanation	For 'Data Structure 1/2', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Polygons exist as a closed, double-linked list / ■ , ■ , ■ , ... , ■ = ■ / 0 1 2 ■ 0 / - The following segments are included / ■ ■ , ■ ■ , ... , ■ ■ = ■ ■ . Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Data Structure 1/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Polygons exist as a closed, double-linked list / ■ , ■ , ■ , ... , ■ = ■ / 0 1 2 ■ 0 / - The following segments are included / ■ ■ , ■ ■ , ... , ■ ■ = ■ ■
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Data Structure 1/2' into a causal sentence instead of repeating the slide title?

Page 46	Data Structure 2/2
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	For 'Data Structure 2/2', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: [Greiner & Hormann, TOG 1998]. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Data Structure 2/2. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Data Structure 2/2'?

Page 47	Proceeding
Source cue	- Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Professor-style explanation	For 'Proceeding', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule). Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Proceeding. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?

Page 48	Pseudo Code
Source cue	while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)
Professor-style explanation	For 'Pseudo Code', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current). Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Pseudo Code. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?

Page 49	Winding Number 1/3
Source cue	- Winding number of a point ■ and a closed curve ■ / indicates the number of times a ray that traverses the entire curve from ■ / winds around ■ / ■ ■, ■ = (cid:3505) ■■ / - Counterclockwise: +1
Professor-style explanation	With 'Winding Number 1/3', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Winding number of a point ■ and a closed curve ■ / indicates the number of times a ray that traverses the entire curve from ■ / winds around ■ / ■ ■, ■ = (cid:3505) ■■ / - Counterclockwise: +1. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.

Technical commentary	This slide is about Winding Number 1/3. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Winding number of a point P and a closed curve C / indicates the number of times a ray that traverses the entire curve from P / winds around C / $WN(P, C) =$ (cid:3505) $WN(P, C)$ / - Counterclockwise: +1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Winding Number 1/3' works per object, per image region, per ray, or per fragment?
Page 50	Winding Number 2/3
Source cue	- The winding number of a point P has the following useful properties / 1. If P or the curve C is continuously changed so that P maintains a / positive distance to C , the winding number will not change / For curve C , the winding number is constant in each region formed by C / 2. When P moves and C crosses once, winding number changes by exactly 1
Professor-style explanation	For 'Winding Number 2/3', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - The winding number of a point P has the following useful properties / 1. If P or the curve C is continuously changed so that P maintains a / positive distance to C , the winding number will not change / For curve C , the winding number is constant in each region formed by C / 2. When P moves and C crosses once, winding number changes by exactly 1. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Winding Number 2/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - The winding number of a point P has the following useful properties / 1. If P or the curve C is continuously changed so that P maintains a / positive distance to C , the winding number will not change / For curve C , the winding number is constant in each region formed by C / 2. When P moves and C crosses once, winding number changes by exactly 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Winding Number 2/3' into a causal sentence instead of repeating the slide title?
Page 51	Winding Number 3/3
Source cue	- From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]

Professor-style explanation	For 'Winding Number 3/3', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Winding Number 3/3. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Winding Number 3/3'?
Page 52	Special Cases
Source cue	- Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge
Professor-style explanation	For 'Special Cases', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Special Cases. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Special Cases' into a causal sentence instead of repeating the slide title?
Page 53	Evaluation

Source cue	- Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,
Professor-style explanation	For 'Evaluation', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Evaluation. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?

Page 54	by exploiting binary outcodes
Source cue	- Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon
Professor-style explanation	For 'by exploiting binary outcodes', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about by exploiting binary outcodes. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'by exploiting binary outcodes'?
Page 55	Literature and other sources used in this chapter
Source cue	The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Professor-style explanation	For 'Literature and other sources used in this chapter', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Literature and other sources used in this chapter. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?
Page 56	Addison-Wesley 2013.
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.
Professor-style explanation	For 'Addison-Wesley 2013.', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Addison-Wesley 2013.. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Addison-Wesley 2013.' into a causal sentence instead of repeating the slide title?

05.1 Cohen-Sutherland Line Clipping

Source location: Section 5.1

Commentary

Cohen-Sutherland assigns outcodes to line endpoints based on which side of the clipping window they lie on. These codes allow quick accept, quick reject, or partial clipping. The algorithm is efficient because many cases are decided before computing intersections.

The exam-friendly idea is the logic of region codes: if both endpoints are inside, keep the segment. If the bitwise AND of the outcodes is nonzero, both endpoints share an outside region and the segment can be rejected. Otherwise, compute an intersection and continue.

Mental model

Outcodes are a cheap classification before doing intersection math.

Check yourself

What does a nonzero bitwise AND of two endpoint outcodes imply?

05.2 Cyrus-Beck Line Clipping

Source location: Section 5.2

Commentary

Cyrus-Beck treats the line parametrically and clips against convex boundaries. Instead of repeatedly using region codes, it finds entering and leaving parameter values along the line.

The key is to reason about the line parameter t . A clipped line segment is not a new unrelated line; it is a restricted interval of the original parametric line. Boundary tests shrink that interval.

Mental model

Clipping a parametric line means narrowing the valid t interval.

Check yourself

Why does Cyrus-Beck naturally require convex clipping regions?

05.3 Sutherland-Hodgman Polygon Clipping

Source location: Section 5.3

Commentary

Sutherland-Hodgman clips a polygon against one boundary at a time. For each edge of the polygon, it decides whether vertices are inside or outside and emits zero, one, or two vertices depending on transitions across the boundary.

This algorithm is easiest to understand as a stream processor. Feed in a polygon, clip it against the left boundary, feed the result into the right boundary, and so on. Each boundary can add intersection vertices.

Mental model

Polygon clipping is repeated edge-by-edge filtering plus intersection insertion.

Check yourself

When can polygon clipping increase the number of vertices?

05.4 Weiler-Atherton and Greiner-Hormann

Source location: Sections 5.4-5.5

Commentary

These algorithms address more complex polygon clipping scenarios, especially when polygon relationships are not as simple as convex-window clipping. They organize intersections and traversal rules to produce correct output boundaries.

For preparation, focus on why simpler algorithms are not enough. Complex polygon intersection can require following alternating boundaries of subject and clipping polygons. That is a topology problem, not only a line-intersection problem.

Mental model

Complex polygon clipping is about traversing boundary networks after intersections are known.

Check yourself

Why are intersection ordering and traversal rules important for polygon clipping?

End-of-lecture summary

If you remember only one thing from Lecture 05, remember this: This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.